

Лабораторная работа № 2. Структуры данных.

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Кортежи	7
3.2	Словари	8
3.3	Множества	9
3.4	Массивы	11
3.5	Задания для самостоятельной работы	18
4	Выводы	29

Список иллюстраций

3.1	Примеры кортежей	7
3.2	Операции над кортежами	8
3.3	Примеры словарей и операций над ними	9
3.4	Примеры множеств и операций над ними	10
3.5	Примеры множеств и операций над ними	11
3.6	Примеры массивов	12
3.7	Примеры массивов, заданных некоторыми функциями через включение	13
3.8	Примеры	15
3.9	Примеры	16
3.10	Примеры	17
3.11	Примеры	18
3.12	Задание 1-2. Работа с множествами и свои примеры работы с ними	19
3.13	Пункт 3.1-3.9	20
3.14	Пункт 3.10-3.11	21
3.15	Пункт 3.11-3.13	22
3.16	Пункт 3.14	23
3.17	Пункт 3.14	24
3.18	Пункт 3.14	25
3.19	Пункт 3.14	26
3.20	Конец пункта 3.14 и задание 4	27
3.21	Задание 5-6	28

Список таблиц

1 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2 Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

3 Выполнение лабораторной работы

3.1 Кортежи

Кортеж (Tuple) — структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы). Синтаксис определения кортежа: (element1, element2, ...)

Рассмотрим примеры кортежей (рис. 3.1):

Примеры кортежей

```
[6]: # пустой кортеж:
      ()

[6]: ()

[7]: # кортеж из элементов типа String:
      favoritelang = ("Python", "Julia", "R")

[7]: ("Python", "Julia", "R")

[8]: # кортеж из целых чисел:
      x1 = (1, 2, 3)

[8]: (1, 2, 3)

[9]: # кортеж из элементов разных типов:
      x2 = (1, 2.0, "tmp")

[9]: (1, 2.0, "tmp")

[10]: # именованный кортеж:
      x3 = (a=2, b=1+2)

[10]: (a = 2, b = 3)
```

Рис. 3.1: Примеры кортежей

Рассмотрим операции над кортежами (рис. 3.2):

Примеры операций над кортежами

```
[11]: # длина кортежа x2:
      length(x2)

[11]: 3

[12]: # обратиться к элементам кортежа x2:
      x2[1], x2[2], x2[3]

[12]: (1, 2.0, "tmp")

[13]: # произвести какую-либо операцию (сложение)
      # с вторым и третьим элементами кортежа x1:
      c = x1[2] + x1[3]

[13]: 5

[14]: # обращение к элементам именованного кортежа x3:
      x3.a, x3.b, x3[2]

[14]: (2, 3, 3)

[15]: # проверка вхождения элементов tmp и 0 в кортеж x2
      # (два способа обращения к методу in()):
      in("tmp", x2), 0 in x2

[15]: (true, false)
```

Рис. 3.2: Операции над кортежами

3.2 Словари

Словарь — неупорядоченный набор связанных между собой по ключу данных. Синтаксис определения словаря: `Dict(key1 => value1, key2 => value2, ...)`

Рассмотрим словари и проведем операции над ними (рис. 3.4):

Словари

```

[16]: # создать словарь с именем phonebook:
phonebook = Dict{"Иванов И.И." => ("867-5309", "333-5544"), "Бухгалтерия" => "555-2368"}

[16]: Dict{String, Any} with 2 entries:
      "Бухгалтерия" => "555-2368"
      "Иванов И.И." => ("867-5309", "333-5544")

[17]: # вывести ключи словаря:
keys(phonebook)

[17]: KeySet for a Dict{String, Any} with 2 entries. Keys:
      "Бухгалтерия"
      "Иванов И.И."

[18]: # вывести значения элементов словаря:
values(phonebook)

[18]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
      "555-2368"
      ("867-5309", "333-5544")

[19]: # вывести заданные в словаре пары "ключ - значение":
pairs(phonebook)

[19]: Dict{String, Any} with 2 entries:
      "Бухгалтерия" => "555-2368"
      "Иванов И.И." => ("867-5309", "333-5544")

[20]: # проверка вхождения ключа в словарь:
haskey(phonebook, "Иванов И.И.")

[20]: true

[21]: # добавить элемент в словарь:
phonebook["Сидоров П.С."] = "555-3344"

[21]: "555-3344"

[23]: # удалить ключ и связанные с ним значения из словаря
pop!(phonebook, "Иванов И.И.")

[23]: ("867-5309", "333-5544")

[24]: # Объединение словарей (функция merge()):
a = Dict{"foo" => 0.0, "bar" => 42.0};
b = Dict{"baz" => 17, "bar" => 13.0};
merge(a, b, merge(b,a))

[24]: (Dict{String, Real}{"bar" => 13.0, "baz" => 17, "foo" => 0.0}, Dict{String, Real}{"bar" => 42.0, "baz" => 17, "foo" => 0.0})

```

Рис. 3.3: Примеры словарей и операций над ними

3.3 Множества

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа. Возможные операции над множествами: объединение, пересечение, разность; принадлежность элемента множеству.

Синтаксис определения множества: `Set([itr])` где `itr` — набор значений, сгенерированных данным итерируемым объектом или пустое множество.

Рассмотрим примеры множеств и операций над ними (рис. 3.4):

Множества

```
[25]: # создать множество из четырёх целочисленных значений:  
A = Set([1, 3, 4, 5])
```

```
[25]: Set{Int64} with 4 elements:  
5  
4  
3  
1
```

```
[26]: # создать множество из 11 символьных значений:  
B = Set("abracadabra")
```

```
[26]: Set{Char} with 5 elements:  
'a'  
'd'  
'r'  
'k'  
'b'
```

```
[27]: # проверка эквивалентности двух множеств:  
S1 = Set([1,2]);  
S2 = Set([3,4]);  
issetequal(S1,S2)
```

```
[27]: false
```

```
[28]: S3 = Set([1,2,2,3,1,2,3,2,1]);  
S4 = Set([2,3,1]);  
issetequal(S3,S4)
```

```
[28]: true
```

```
[29]: # объединение множеств:  
C=union(S1,S2)
```

```
[29]: Set{Int64} with 4 elements:  
4  
2  
3  
1
```

```
[30]: # пересечение множеств:  
D = intersect(S1,S3)
```

```
[30]: Set{Int64} with 2 elements:  
2  
1
```

Рис. 3.4: Примеры множеств и операций над ними

```

[31]: # разность множеств:
      E = setdiff(S3,S1)

[31]: Set{Int64} with 1 element:
      3

[32]: # проверка вхождения элементов одного множества в другое:
      issubset(S1,S4)

[32]: true

[33]: # добавление элемента в множество:
      push!(S4, 99)

[33]: Set{Int64} with 4 elements:
      2
      99
      3
      1

[34]: # удаление последнего элемента множества:
      pop!(S4)

[34]: 2

```

Рис. 3.5: Примеры множеств и операций над ними

3.4 Массивы

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов. Общий синтаксис одномерных массивов:

```

array_name_1 = [element1, element2, ...]
array_name_2 = [element1 element2 ...]

```

Рассмотрим примеры массивов (рис. 3.6):

Массивы

```
[35]: # создание пустого массива с абстрактным типом:
empty_array_1 = []

[35]: Any[]

[37]: # создание пустого массива с конкретным типом:
empty_array_2 = (Int64)[]

[37]: Int64[]

[38]: empty_array_3 = (Float64)[]

[38]: Float64[]

[39]: # вектор-столбец:
a = [1, 2, 3]

[39]: 3-element Vector{Int64}:
      1
      2
      3

[40]: # вектор-строка:
b = [1 2 3]

[40]: 1x3 Matrix{Int64}:
      1  2  3

[41]: # многомерные массивы (матрицы):
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]

[41]: 3x3 Matrix{Int64}:
      1  4  7
      2  5  8
      3  6  9

[42]: B = [[1 2 3]; [4 5 6]; [7 8 9]]

[42]: 3x3 Matrix{Int64}:
      1  2  3
      4  5  6
      7  8  9

[45]: # одномерный массив из 8 элементов (массив $1 \times 8$)
# со значениями, случайно распределёнными на интервале [0, 1]:
c = rand(1,8)

[45]: 1x8 Matrix{Float64}:
      0.555361  0.188253  0.902527  0.810364 ... 0.981847  0.118177  0.111058
```

Рис. 3.6: Примеры массивов

Рассмотрим примеры массивов, заданных некоторыми функциями через включение (рис. 3.7):

```

[49]: # многомерный массив $2 \times 3$ (2 строки, 3 столбца) элементов
      # со значениями, случайно распределёнными на интервале [0, 1]:
      C = rand(2,3)

[49]: 2x3 Matrix{Float64}:
      0.900034  0.852383  0.279751
      0.0869465 0.122167  0.542902

[50]: # трёхмерный массив:
      D = rand(4, 3, 2)

[50]: 4x3x2 Array{Float64, 3}:
     [:, :, 1] =
      0.349258  0.20746  0.209247
      0.878426  0.92443  0.793306
      0.987893  0.080608  0.527845
      0.596582  0.502803  0.393346

     [:, :, 2] =
      0.309682  0.961072  0.639578
      0.855543  0.72326  0.803434
      0.799612  0.0782093 0.261948
      0.0732649 0.408365  0.0923995

[51]: # массив из квадратных корней всех целых чисел от 1 до 10:
      roots = [sqrt(i) for i in 1:10]

[51]: 10-element Vector{Float64}:
      1.0
      1.4142135623730951
      1.7320508075688772
      2.0
      2.23606797749979
      2.449489742783178
      2.6457513110645907
      2.8284271247461903
      3.0
      3.1622776601683795

[52]: # массив с элементами вида 3*x^2,
      # где x - нечётное число от 1 до 9 (включительно)
      ar_1 = [3*i^2 for i in 1:2:9]

[52]: 5-element Vector{Int64}:
      3
      27
      75
      147
      243

[53]: # массив квадратов элементов, если квадрат не делится на 5 или 4:
      ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

[53]: 4-element Vector{Int64}:
      1
      9
      49
      81

```

Рис. 3.7: Примеры массивов, заданных некоторыми функциями через включение

Некоторые операции для работы с массивами:

- `length(A)` — число элементов массива A;
- `ndims(A)` — число размерностей массива A;
- `size(A)` — кортеж размерностей массива A;
- `size(A, n)` — размерность массива A в заданном направлении;

- `copy(A)` — создание копии массива `A`;
- `ones()`, `zeros()` — создать массив с единицами или нулями соответственно;
- `fill(value,array_name)` — заполнение массива заранее определенным значением;
- `sort()` — сортировка элементов;
- `collect()` — вернуть массив всех элементов в коллекции или итераторе;
- `reshape()` — изменение размера массива;
- `transpose()` — транспонирование массива;

Рассмотрим несколько примеров (рис. 3.8):

Некоторые операции для работы с массивами

```
[55]: # одномерный массив из пяти единиц:
      ones(5)

[55]: 5-element Vector{Float64}:
      1.0
      1.0
      1.0
      1.0
      1.0

[56]: # двумерный массив 2x3 из единиц:
      ones(2,3)

[56]: 2x3 Matrix{Float64}:
      1.0  1.0  1.0
      1.0  1.0  1.0

[57]: # одномерный массив из 4 нулей:
      zeros(4)

[57]: 4-element Vector{Float64}:
      0.0
      0.0
      0.0
      0.0

[58]: # заполнить массив 3x2 цифрами 3.5
      fill(3.5,(3,2))

[58]: 3x2 Matrix{Float64}:
      3.5  3.5
      3.5  3.5
      3.5  3.5

[59]: # заполнение массива посредством функции repeat():
      repeat([1,2],3,3)
      repeat([1 2],3,3)

[59]: 3x6 Matrix{Int64}:
      1  2  1  2  1  2
      1  2  1  2  1  2
      1  2  1  2  1  2

[60]: # преобразование одномерного массива из целых чисел от 1 до 12
      # в двумерный массив 2x6
      a = collect(1:12)
      b = reshape(a,(2,6))

[60]: 2x6 Matrix{Int64}:
      1  3  5  7  9  11
      2  4  6  8  10 12

[61]: # транспонирование
      b

[61]: 2x6 Matrix{Int64}:
      1  3  5  7  9  11
      2  4  6  8  10 12
```

Рис. 3.8: Примеры

```
[62]: # транспонирование
c = transpose(b)

[62]: 6x2 transpose(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12
```

```
[63]: # массив 10x5 целых чисел в диапазоне [10, 20]:
ar = rand(10:20, 10, 5)
```

```
[63]: 10x5 Matrix{Int64}:
      11 17 19 14 20
      12 16 16 10 11
      18 18 19 18 13
      12 13 11 12 19
      18 11 10 10 10
      10 11 16 10 19
      20 13 10 15 19
      18 13 20 20 20
      17 17 16 19 19
      13 18 19 19 19
```

```
[64]: # выбор всех значений строки в столбце 2:
ar[:, 2]
```

```
[64]: 10-element Vector{Int64}:
      17
      16
      18
      13
      11
      11
      13
      13
      17
      18
```

```
[65]: # выбор всех значений в столбцах 2 и 5:
ar[:, [2, 5]]
```

```
[65]: 10x2 Matrix{Int64}:
      17 20
      16 11
      18 13
      13 19
      11 10
      11 19
      13 19
      13 20
      17 19
      18 19
```

Рис. 3.9: Примеры


```

[66]: # все значения строк в столбцах 2, 3 и 4:
      ar[:, 2:4]

[66]: 10x3 Matrix{Int64}:
      17 19 14
      16 16 10
      18 19 18
      13 11 12
      11 10 10
      11 16 10
      13 10 15
      13 20 20
      17 16 19
      18 19 19

[67]: # значения в строках 2, 4, 6 и в столбцах 1 и 5:
      ar[[2, 4, 6], [1, 5]]

[67]: 3x2 Matrix{Int64}:
      12 11
      12 19
      10 19

[68]: # значения в строке 1 от столбца 3 до последнего столбца:
      ar[1, 3:end]

[68]: 3-element Vector{Int64}:
      19
      14
      20

[70]: # сортировка по столбцам:
      sort(ar, dims=1)

[70]: 10x5 Matrix{Int64}:
      10 11 10 10 10
      11 11 10 10 11
      12 13 11 10 13
      12 13 16 12 19
      13 13 16 14 19
      17 16 16 15 19
      18 17 19 18 19
      18 17 19 19 19
      18 18 19 19 20
      20 18 20 20 20

[71]: # сортировка по строкам:
      sort(ar, dims=2)

[71]: 10x5 Matrix{Int64}:
      11 14 17 19 20
      10 11 12 16 16
      13 18 18 18 19
      11 12 12 13 19
      10 10 10 11 18
      10 10 11 16 19
      10 13 15 19 20
      13 18 20 20 20
      16 17 17 19 19
      13 18 19 19 19

```

Рис. 3.10: Примеры

```

[72]: # поэлементное сравнение с числом
      # (результат - массив логических значений):
      ar .> 14

[72]: 10x5 BitMatrix:
      0 1 1 0 1
      0 1 1 0 0
      1 1 1 1 0
      0 0 0 0 1
      1 0 0 0 0
      0 0 1 0 1
      1 0 0 1 1
      1 0 1 1 1
      1 1 1 1 1
      0 1 1 1 1

[73]: # возврат индексов элементов массива, удовлетворяющих условию:
      findall(ar .> 14)

[73]: 29-element Vector{CartesianIndex{2}}:
      CartesianIndex(3, 1)
      CartesianIndex(5, 1)
      CartesianIndex(7, 1)
      CartesianIndex(8, 1)
      CartesianIndex(9, 1)
      CartesianIndex(1, 2)
      CartesianIndex(2, 2)
      CartesianIndex(3, 2)
      CartesianIndex(9, 2)
      CartesianIndex(10, 2)
      CartesianIndex(1, 3)
      CartesianIndex(2, 3)
      CartesianIndex(3, 3)
      :
      CartesianIndex(3, 4)
      CartesianIndex(7, 4)
      CartesianIndex(8, 4)
      CartesianIndex(9, 4)
      CartesianIndex(10, 4)
      CartesianIndex(1, 5)
      CartesianIndex(4, 5)
      CartesianIndex(6, 5)
      CartesianIndex(7, 5)
      CartesianIndex(8, 5)
      CartesianIndex(9, 5)
      CartesianIndex(10, 5)

```

Рис. 3.11: Примеры

3.5 Задания для самостоятельной работы

1. Даны множества: $A = 0, 3, 4, 9, B = 1, 3, 4, 7, C = 0, 1, 2, 4, 7, 8, 9$. Найдем $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$
2. Приведем свои примеры с выполнением операций над множествами элементов разных типов (рис. 3.12):

Задание для самостоятельного выполнения

```
[74]: # 1. Операции над множествами
A = Set{0, 3, 4, 9}
B = Set{1, 3, 4, 7}
C = Set{0, 1, 2, 4, 7, 8, 9}

[74]: Set{Int64} with 7 elements:
 0
 4
 7
 2
 9
 8
 1

[78]: P = union(intersect(A, B), intersect(A, C), intersect(B, C))

[78]: Set{Int64} with 6 elements:
 0
 4
 7
 9
 3
 1

[79]: # 2. Примеры операций над множествами разных типов
# Множество строк
set1 = Set{"apple", "banana", "orange"}

[79]: Set{String} with 3 elements:
"orange"
"banana"
"apple"

[80]: # Множество чисел
set2 = Set{1, 2, 3, 4, 5}

[80]: Set{Int64} with 5 elements:
 5
 4
 2
 3
 1

[81]: # Объединение множеств разных типов (нельзя смешивать типы в Julia)
# Поэтому создадим множество Any
set_mixed = Set{Any}{"text", 42, 3.14, :symbol}

[81]: Set{Any} with 4 elements:
:symbol
3.14
42
"text"

[82]: println("Смешанное множество: ", set_mixed)

Смешанное множество: Set{Any{::Symbol, 3.14, 42, "text"}}
```

Рис. 3.12: Задание 1-2. Работа с множествами и свои примеры работы с ними

3. Создадим разными способами массивы и векторы (рис. 3.13):

```
# 3. Создание массивов различными способами
# 3.1 Массив (1, 2, 3, ..., N), N > 20
N = 25
arr1 = collect(1:N) # Преобразуем диапазон в массив
println("3.1:", arr1)

3.1: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25]

84]: # 3.2 Массив (N, N-1, ..., 2, 1)
arr2 = collect(N:-1:1) # Обратный диапазон
println("3.2:", arr2)

3.2: [25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

87]: # 3.3 Массив (1, 2, 3, ..., N-1, N, N-1, ..., 2, 1)
arr3 = vcat(1:N, N:-1:1) # Объединяем два диапазона
println("3.3:", arr3)

3.3: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

86]: # 3.4 Массив tmp = {4, 6, 3}
tmp = [4, 6, 3]
println("3.4 tmp:", tmp)

3.4 tmp: [4, 6, 3]

87]: # 3.5 Первый элемент tmp повторится 10 раз
arr5 = fill(tmp[1], 10) # fill создает массив с повторяющимся значением
println("3.5:", arr5)

3.5: [4, 4, 4, 4, 4, 4, 4, 4, 4, 4]

88]: # 3.6 Все элементы tmp повторятся 10 раз
arr6 = repeat(tmp, 10) # repeat повторит весь массив
println("3.6:", arr6)

3.6: [4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]

89]: # 3.7 Первый элемент 11 раз, второй и третий - по 10 раз
arr7 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
println("3.7:", arr7)

3.7: [4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]

90]: # 3.8 Первый элемент 10 раз подряд, второй - 20, третий - 30
arr8 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
println("3.8:", arr8)

3.8: [4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]

91]: # 3.9 2^i[tmp[i]] где i=1,2,3, и 2^i[tmp[i]] повторится 4 раза
# Создан массив: 2^1[tmp[1]], 2^2[tmp[2]], и 2^3[tmp[3]] повторится 4 раза
arr9 = vcat(2^tmp[1], 2^tmp[2], fill(2^tmp[3], 4))
println("3.9 Массив:", arr9)

3.9 Массив: [16, 64, 8, 8, 8, 8]

92]: # Считаем сколько раз встречается цифра 6
count_6 = count(x -> x == 6, arr9)
println("3.9 Количество цифр 6:", count_6)

3.9 Количество цифр 6: 0
```

Рис. 3.13: Пункт 3.1-3.9

```

[95]: using Statistics

[180]: # 3.10 Вектор  $y = e^x \cdot \cos(x)$  для  $x = 3, 3.1, 3.2, \dots, 6$ 
x_range = 3:0.1:6 # Диапазон от 3 до 6 с шагом 0.1
y = exp(x_range) .* cos(x_range) # Поэлементные операции

[180]: 31-element Vector{Float64}:
 -19.884530844146987
 -22.178753389342127
 -24.490696732801293
 -26.77318244299338
 -28.969237768093574
 -31.011186439374516
 -32.819774760338504
 -34.30336011037369
 -35.35719361853035
 -35.86283371230767
 -35.68773248011913
 -34.68504225166807
 -32.693695428321746
  :
 25.046704998273004
 42.09920106253839
 61.99663027669454
 84.92906736250268
111.0615860420258
140.5250750527875
173.40577640857734
209.73349424783467
249.46844055885668
292.4867067371223
338.5643778585117
387.36034029093076

[100]: mean_y = mean(y) # Среднее значение
println("3.10 Среднее значение y: ", round(mean_y, digits=4))
3.10 Среднее значение y: 53.1137

[181]: # 3.11 Вектор  $(x^i, y^j)$  для заданных параметров
x_vals = 0.1
y_vals = 0.2
i_range = 3:3:36 #  $i = 3, 6, 9, \dots, 36$ 

[181]: 3:3:36

[182]: j_range = 1:3:34 #  $j = 1, 4, 7, \dots, 34$ 

[182]: 1:3:34

[183]: # Создаем массивы  $x^i$  и  $y^j$ 
x_powers = [x^i for x in x_vals, i in i_range]

[183]: 12-element Vector{Float64}:
 0.0010000000000000002
 1.0000000000000004e-6
 1.0000000000000005e-9
 1.0000000000000008e-12
 1.0000000000000009e-15
 1.0000000000000008e-18
 1.0000000000000012e-21
 1.0000000000000012e-24
 1.0000000000000015e-27
 1.0000000000000017e-30
 1.0000000000000018e-33
 1.000000000000002e-36

```

Рис. 3.14: Пункт 3.10-3.11

```

[184]: y_powers = [y**j for y in y_vals, j in j_range]

[184]: 12-element Vector{Float64}:
 0.2
 0.0016000000000000003
 1.2800000000000000e-5
 1.0240000000000000e-7
 8.192000000000000e-10
 6.553600000000000e-12
 5.242880000000000e-14
 4.194304000000000e-16
 3.355443200000000e-18
 2.684354560000000e-20
 2.147483648000000e-22
 1.717986918400000e-24

[185]: arr11 = vcat(x_powers[:,1], y_powers[:,1])
println("3.11 Длина вектора: ", length(arr11), ", уникальные значения: ", unique(arr11))

3.11 Длина вектора: 24, уникальные значения: [0.0016000000000000002, 1.000000000000000e-5, 1.000000000000000e-9, 1.000000000000000e-12, 1.000000000000000e-15, 1.000000000000000e-18, 1.000000000000000e-21, 1.000000000000000e-24, 1.000000000000000e-27, 1.000000000000000e-30, 1.000000000000000e-33, 1.000000000000000e-36, 0.2, 0.0016000000000000003, 1.280000000000000e-5, 1.024000000000000e-7, 8.192000000000000e-10, 6.553600000000000e-12, 5.242880000000000e-14, 4.194304000000000e-16, 3.355443200000000e-18, 2.684354560000000e-20, 2.147483648000000e-22, 1.717986918400000e-24]

[188]: # 3.12 Вектор x/i для i = 1,2,...,M, M=25
arr12 = [2^i / i for i in 1:25];
println(arr12)

[2.0, 2.0, 2.6666666666666665, 4.0, 6.4, 10.666666666666666, 18.285714285714285, 32.0, 56.888888888888886, 102.4, 186.1818181818182, 341.3333333333333, 630.1538461538462, 1170.2857142857142, 2184.5333333333333, 4096.0, 7710.117647058823, 14563.555555555555, 27594.105263157893, 52428.8, 99864.38095238095, 190650.18181818182, 364722.0869565217, 699050.6666666666, 1.34217728e6]

[111]: # 3.13 Вектор ("fn1", "fn2", ..., "fnN"), N=30
N_names = 30
arr13 = ["fn$i" for i in 1:N_names] # Генератор строк

[111]: 30-element Vector{String}:
 "fn1"
 "fn2"
 "fn3"
 "fn4"
 "fn5"
 "fn6"
 "fn7"
 "fn8"
 "fn9"
 "fn10"
 "fn11"
 "fn12"
 "fn13"
 ⋮
 "fn19"
 "fn20"
 "fn21"
 "fn22"
 "fn23"
 "fn24"
 "fn25"
 "fn26"
 "fn27"
 "fn28"
 "fn29"
 "fn30"

```

Рис. 3.15: Пункт 3.11-3.13

```

[116]: using Random
      using Primes

[193]: # 3.14) Создание случайных векторов
      Random.seed!(123) # Для воспроизводимости результатов
      n = 250
      x = rand(0:999, n); # Вектор x из случайных целых от 0 до 999

[194]: y = rand(0:999, n); # Вектор y из случайных целых от 0 до 999

[128]: # Вектор 1
      vec1 = [y[i+1] - x[i] for i in 1:n-1]

[128]: 249-element Vector{Int64}:
      -211
      -124
      -794
      460
      228
      521
      626
      -11
      187
      328
      298
      -582
      807
      ⋮
      -183
      776
      305
      286
      -168
      -29
      610
      -132
      -104
      576
      767
      531

[130]: # Вектор 2
      vec2 = [x[i] + 2*x[i+1] - x[i+2] for i in 1:n-2]

[130]: 248-element Vector{Int64}:
      803
      2176
      745
      859
      1261
      -455
      1330
      1766
      708
      543
      2158
      451
      608
      ⋮
      490
      740

```

Рис. 3.16: Пункт 3.14

```

[132]: # Вектор 3
vec3 = [sin(y[i]) / cos(x[i+1]) for i in 1:n-1]

[132]: 249-element Vector{Float64}:
 8.571958432842645
-1.4219188205712436
-2.787213484786365
-1.0486185570849187
 0.33723681825489156
-0.831871621143289
 0.861917879898797966
 2.8284784845648342
-1.837142855647466
-2.8282182658162984
 1.059184212182359
-1.1778474388677485
 0.4524425273746784
  ⋮
-1.811298872243521
 2.8524161668963367
-1.9846223874189785
-2.1398443621166186
-0.9687237382792472
-0.4288423819612119
-0.8489412571423979
-0.7816361216795618
-2.1482721684187494
-0.5657783750795751
-0.4144595582368843
 0.88879814793338226

[134]: # Сумма
sum_val = sum(exp(-x[i+1]) / (x[i] + 10) for i in 1:n-1)

[134]: 0.00817065218565897441

[136]: # Элементы вектора y > 600
y_greater_600 = y[y .> 600]

[136]: 114-element Vector{Int64}:
 650
 753
 911
 678
 922
 767
 655
 824
 847
 905
 823
 708
 679
  ⋮
 978
 999
 982
 945
 888
 666
 695
 928
 736
 766
 842
 922

[137]: println("5. Элементы y > 600: ", length(y_greater_600), " элементов")

5. Элементы y > 600: 114 элементов

*[138.. println("Первые 10: ", y_greater_600[1:min(10, end)])

Первые 10: [650, 753, 911, 678, 922, 767, 655, 824, 847, 905]

```

Рис. 3.17: Пункт 3.14


```

[139]: # Индексы элементов y > 600
indices_y_600 = findall(y .> 600)

[139]: 114-element Vector{Int64}:
        5
        6
        7
        8
        9
       10
       11
       12
       14
       17
       21
       22
       23
       :
      227
      230
      232
      233
      235
      236
      238
      240
      245
      247
      249
      250

[141]: # Значения x, соответствующие y > 600
x_corresponding = x[y .> 600]

[141]: 114-element Vector{Int64}:
       525
       390
        44
       933
       580
       327
       526
       836
       465
       720
       276
       983
       880
       :
        95
       858
       664
       834
       463
       321
       136
       200
       222
        2
       391
       885

[143]: # Вектор 4
x_mean = mean(x)
vec4 = sqrt.(abs.(x .- x_mean))

[143]: 250-element Vector{Float64}:
  5.741428393701344
  9.897676495016393
 20.049039877260956
 17.263719182146122
  6.079802628375365
  9.901313044238123
 21.072161730586636

```

Рис. 3.18: Пункт 3.14

```

[145]: # Элементы y, отстоящие от максимума не более чем на 200
y_max = maximum(y)

[145]: 999

[146]: y_close_to_max = y[abs.(y .- y_max) .<= 200]

[146]: 57-element Vector{Int64}:
 911
 922
 824
 847
 905
 823
 952
 817
 901
 820
 807
 911
 990
  :
 894
 853
 872
 804
 978
 999
 902
 945
 808
 928
 842
 922

[147]: println("9. Элементы y, отстоящие от максимума ≤ 200: ", length(y_close_to_max))
9. Элементы y, отстоящие от максимума ≤ 200: 57

[149]: # Чётные и нечётные элементы x
even_count = count(iseven, x)

[149]: 129

[150]: odd_count = count(isodd, x)

[150]: 121

[151]: println("10. Чётные элементы x: ", even_count)
10. Чётные элементы x: 129

*[152]: println("10. Нечётные элементы x: ", odd_count)
Нечётные элементы x: 121

[153]: # Элементы x, кратные 7
multiples_of_7 = count(x -> x % 7 == 0, x)

[153]: 30

[154]: println("11. Элементы x, кратные 7: ", multiples_of_7)
11. Элементы x, кратные 7: 30

[155]: # Сортировка x в порядке возрастания y
x_sorted_by_y = x[sortperm(y)]

[155]: 250-element Vector{Int64}:
 471
 799
 152
 34
 100

```

Рис. 3.19: Пункт 3.14

4. Создадим массив `squares`, в котором будут храниться квадраты всех целых чисел от 1 до 100 (рис. 3.20):

```

[157]: # Top-10 наибольших элементов x
x_top10 = sort(x, rev=true)[1:min(10, end)]

[157]: 10-element Vector{Int64}:
 996
 993
 993
 986
 983
 974
 959
 953
 949
 943

[159]: # Уникальные элементы x
x_unique = unique(x)

[159]: 227-element Vector{Int64}:
 521
 586
 890
 190
 525
 390
 44
 933
 580
 327
 526
 836
 40
 ⋮
 136
 152
 200
 492
 371
 126
 222
 870
 2
 75
 391
 885

[160]: println("14. Уникальные элементы x: ", length(x_unique), " элементов")
14. Уникальные элементы x: 227 элементов

[161]: # 4. Массив квадратов чисел от 1 до 100
squares = [i^2 for i in 1:100]

[161]: 100-element Vector{Int64}:
 1
 4
 9
 16
 25
 36
 49
 64
 81
 100
 121
 144
 169
 ⋮
 7921

```

Рис. 3.20: Конец пункта 3.14 и задание 4

5. Подключим пакет Primes (функции для вычисления простых чисел). Сгенерируем массив `myprimes`, в котором будут храниться первые 168 простых чисел. Определим 89-е наименьшее простое число. Получим срез массива с 89-го до 99-го элемента включительно, содержащий наименьшие простые числа (рис. 3.21)
6. Вычислим выражения (рис. 3.21):

```

[163]: # 5. Простые числа с использованием пакета Primes
myprimes = primes(1000)[1:168] # Первые 168 простых чисел

[163]: 168-element Vector{Int64}:
      2
      3
      5
      7
     11
     13
     17
     19
     23
     29
     31
     37
     41
      :
    919
    929
    937
    941
    947
    953
    967
    971
    977
    983
    991
    997

* [165]: println("89-е наименьшее простое число: ", myprimes[89])
      89-е наименьшее простое число: 461

* [166]: println("Срез с 89-го до 99-го: ", myprimes[89:99])
      Срез с 89-го до 99-го: [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]

* [167]: # 6. Вычисления сумм
      # 6.1) i=1 до 100
      sum1 = sum(i^3 + 4*i^2 for i in 1:100)

[167]: 26855900

[189]: # 6.2) i=1 до M=25
      M = 25
      sum2 = sum(2*i/i + 3*i/i^2 for i in 1:M)

[189]: 2.1291704368143802e9

[190]: # 6.3) Сложная сумма
      sum6 = 1
      num = 1

[190]: 1

[191]: for i in 2:2:38
      num *= i/(i+1)
      sum6 += num
      end

[192]: print(sum6)
      6.976346137897618

[ ]:

```

Рис. 3.21: Задание 5-6

4 Выводы

В процессе выполнения данной лабораторной работы я изучила несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.